

Interview Prep — Study Order

Eight topic documents, 79 interview questions plus 24 follow-ups, roughly 34,500 words total. This file exists to answer one question — what do I read first?

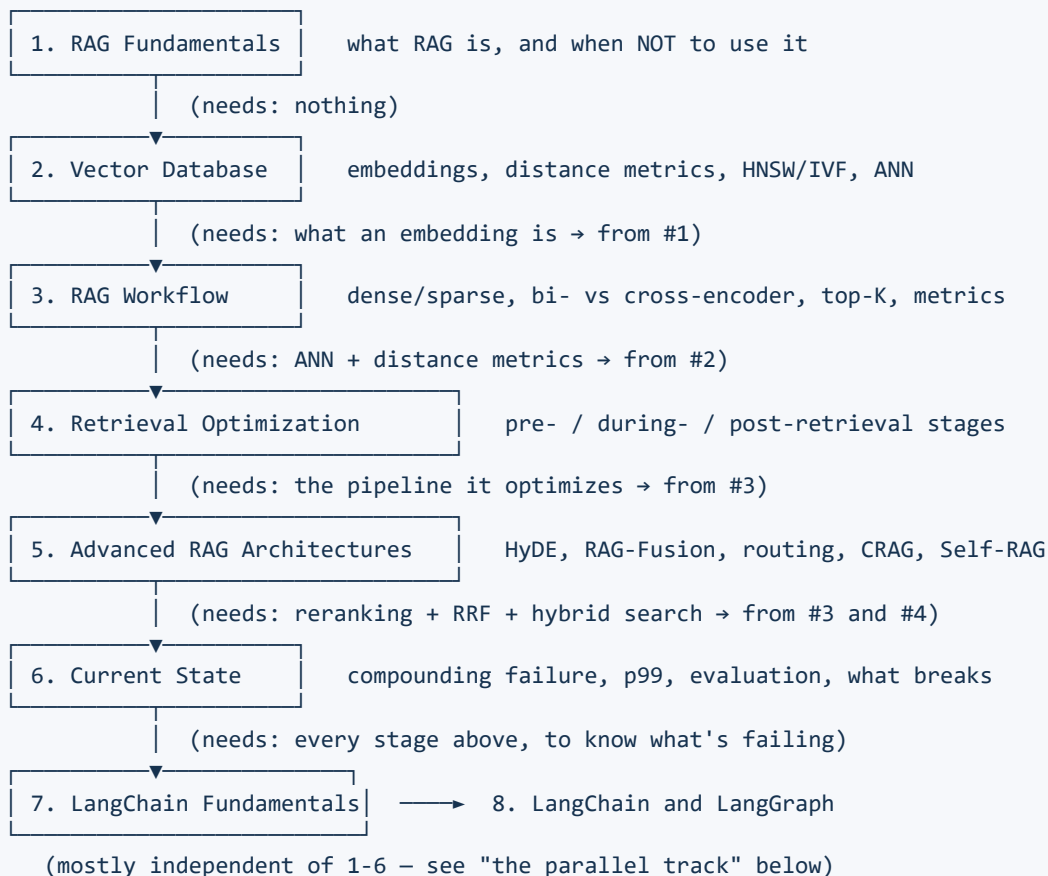
The short version: **do not read these alphabetically, and do not start with the most impressive-sounding one.** Each document assumes you already have specific vocabulary from the ones before it. Read out of order and you'll hit answers that casually say "the cross-encoder reranks the RRF-fused candidates" before anything has told you what any of those three words mean.

The order, at a glance

#	Document	Q + follow-ups	Time	Why here
1	RAG Fundamentals	9 + 3	~35 min	The conceptual bedrock. Also where the single most-asked opener lives.
2	Vector Database	11 + 3	~45 min	The storage layer everything downstream assumes exists.
3	RAG Workflow	10 + 4	~30 min	Retrieval mechanics end to end — the pipeline itself.
4	Retrieval Optimization Techniques	9 + 3	~35 min	Now that you know the pipeline, how to make each stage better.
5	Advanced RAG Architectures	9 + 4	~45 min	The named patterns built on top of optimized retrieval.
6	Current State of RAG	9 + 1	~20 min	What breaks in production. The senior-level differentiator.
7	LangChain Fundamentals	9 + 3	~40 min	The framework, once you know what you're framing.
8	LangChain and LangGraph	13 + 3	~40 min	Agents, LCEL, stateful graphs — the deepest framework material.

One focused pass: roughly 5 hours. Realistically spread over 3-4 sessions, not one sitting.

Why this order — the dependency chain



Phase 1 — Foundations (docs 1-3)

Start with RAG Fundamentals even if it feels too basic. It's where "*RAG vs. fine-tuning vs. long-context*" lives, and that is the single most common opening question in a RAG interview. Getting it crisp — with the cost math — sets the tone for everything after. It also covers when RAG is the *wrong* answer, which is the kind of thing that separates a considered engineer from someone reciting a pipeline.

Vector Database next, because docs 3-6 all quietly assume you know what ANN, HNSW, IVF, and cosine-vs-dot-product mean. This is the longest doc in the set and the most vocabulary-dense — budget real time for it.

Then RAG Workflow, which walks the retrieval pipeline end to end. With #1 and #2 behind you, this reads fast: it's mostly assembling pieces you already have names for.

Phase 2 — Making retrieval actually good (docs 4-5)

Retrieval Optimization Techniques reframes everything from #3 into three intervention points — before, during, and after the search. This framing is genuinely useful in an interview: when someone asks "retrieval is bad, what do you do," answering by *stage* sounds structured rather than like a list of tricks.

Advanced RAG Architectures is the named-pattern layer — HyDE, RAG-Fusion, step-back prompting, routing, CRAG, Self-RAG. Do it after #4, because several of its answers lean on RRF and reranking being already familiar.

Phase 3 — Production reality (doc 6)

Current State of RAG is the shortest document here and the highest-value one for a 4-YOE interview — it's where failure modes, p99 latency, and evaluation strategy live, which is exactly the territory where seniority shows.

But it deliberately comes sixth, not first. Its opening question is "*if each stage is 90% reliable, why isn't the system 90% reliable?*" — that only means something once you can name the six stages. Reading it early feels impressive and teaches you very little.

Phase 4 — The framework track (docs 7-8)

LangChain Fundamentals, then **LangChain and LangGraph**. These two are a genuinely separate axis: *how you build it*, not *how retrieval works*. If the job description mentions LangChain or agents, treat this phase as mandatory and possibly move it earlier. If it doesn't, this is the phase to drop when you're short on time.

Within the pair the order is non-negotiable, though — #8 opens on chain-vs-agent and LCEL migration, both of which assume the component vocabulary from #7.

The parallel track

Docs 7-8 don't depend on docs 1-6 for anything except a passing familiarity with retrievers. If you're prepping over several days, running the RAG track and the LangChain track in parallel works fine — a RAG doc in the morning, a LangChain doc in the evening. What does *not* work is interleaving *within* a track.

Common ordering mistakes

- **Starting with Current State of RAG** because it sounds the most senior. Its whole value is diagnostic, and you can't diagnose a pipeline you can't yet name.
- **Skipping Vector Database** because "I'll pick up HNSW from context." You won't — it's the load-bearing vocabulary for four later documents.
- **Doing LangChain first** because it's the thing you've actually written code in. Framework fluency without the retrieval concepts underneath reads as tool-familiarity, not engineering judgment.
- **Reading Advanced RAG Architectures before Retrieval Optimization.** They look interchangeable from the outside; they aren't. #5 assumes #4's fusion and reranking material.

If you're short on time

One evening (~1.5 hrs) — the highest-signal core: 1. RAG Fundamentals (Q1-Q4 only: what RAG is, vs. fine-tuning, vs. long context, hallucination) 2. RAG Workflow (all of it — it's the shortest path to sounding fluent about the pipeline) 3. Current State of RAG (Q1, Q7, Q8 — compounding reliability, joint evaluation, production non-negotiables)

A weekend (~5 hrs) — the full order above, in four sittings: 1+2 / 3+4 / 5+6 / 7+8.

Night before — don't read anything new. Re-read only these five, out loud: - RAG Fundamentals Q2 (RAG vs. fine-tuning — the decision rule) - Vector Database Q2 (cosine vs. dot vs. Euclidean) - RAG Workflow Q7 (retrieval metrics — Precision/Recall/MRR/NDCG) - Current State of RAG Q1 (compounding reliability, the $0.9^6 \approx 53\%$ math) - Current State of RAG Q6 (p99 latency and its segments)

How to actually use each document

Every answer in every doc is written in two layers on purpose:

-  **Strong answer** — plain language with an everyday example. This is the one to *understand*.
-  **Standard Interview Answer** — jargon-precise and compressed. This is the one to *say*.

The intended loop is: read the question, answer it out loud from memory first, *then* read the  answer to check your understanding, and finally read the  version to pick up the exact terminology. Reading the answers without attempting them first is the most common way to feel prepared and not be.

The  **follow-ups** are where the real interview usually goes — they're the push-back a good interviewer gives after an adequate first answer. If you're tight on time, skipping a main question hurts less than skipping its follow-up.